

ui.toggle 全面详解（基于 NiceGUI 文档）

ui.toggle 是 NiceGUI 中用于实现二选一（布尔值）或多选项切换的核心交互组件，基于 Quasar 的 QToggle 组件实现。它支持布尔切换、多选项分组、自定义图标 / 颜色 / 样式，且具备数据绑定、状态监听等高级能力，适用于开关控制、选项选择、功能启用等场景。以下从核心特性、基础用法、高级功能等维度展开全面解析。

一、核心基础

1. 组件本质与核心特性

- 底层依赖：基于 Quasar 的 QToggle 组件，继承其成熟的切换交互逻辑、样式体系和无障碍支持。
- 核心功能：支持「布尔切换」（True/False）和「多选项分组」（radio 模式），可通过图标、颜色、文本定制外观，支持双向数据绑定和状态变化监听。
- 关键差异：与 ui.switch 相比，ui.toggle 更灵活支持多选项分组和图标定制；与 ui.radio 相比，ui.toggle 样式更紧凑，支持布尔值单独使用。

2. 初始化参数（核心配置）

参数名	类型	说明	默认值
text	str	切换组件的文本标签（显示在开关 / 选项旁）	""
value	bool / str / int	初始值（布尔值用于单独切换，字符串 / 整数用于分组选项）	False
on_change	Callable	值变化时触发的回调函数（e.value 为当前值）	-
group	str / None	分组名称（相同组名的 toggle 自动形成互斥选择，即 radio 模式）	None
icon	str / None	开关 / 选项左侧显示的图标名称（符合 Quasar 图标规范）	None
color	str / None	激活状态的颜色（支持 Quasar 颜色、Tailwind 颜色、CSS 颜色）	"primary"
label_position	str	文本标签位置（可选值："left"、"right"、"top"、"bottom"）	"right"
dense	bool	是否使用紧凑模式（减少组件高度）	False
disabled	bool	是否禁用组件（禁用后不可交互，视觉灰度）	False

二、基础使用示例

1. 核心用法全覆盖（布尔切换 + 分组选择）

展示单独布尔切换、多选项分组、自定义图标 / 颜色 / 标签位置等基础用法：

```
from nicegui import ui

with ui.column().classes('gap-4'):
    # 1. 单独布尔切换（开关功能，绑定回调）
    ui.toggle('Enable feature', on_change=lambda e: ui.notify(f'Enabled: {e.value}'))

    # 2. 多选项分组（radio 模式，互斥选择）
    ui.label('Select theme:')
    with ui.row().classes('gap-4'):
        ui.toggle('Light', value='light', group='theme', on_change=lambda e:
        ui.notify(f'Theme: {e.value}'))
        ui.toggle('Dark', value='dark', group='theme')
        ui.toggle('System', value='system', group='theme')

    # 3. 自定义图标+颜色+标签位置
    ui.toggle(
        'Notifications',
        icon='notifications',
        color='red',
        label_position='left',
        value=True # 初始激活状态
    )

    # 4. 紧凑模式+禁用状态
    with ui.row().classes('gap-4'):
        ui.toggle('Dense mode', dense=True)
        ui.toggle('Disabled', disabled=True)

ui.run()
```

效果：

- 第一个 toggle 为单独布尔切换，点击切换启用 / 禁用状态，触发通知；
- 中间三个 toggle 为同一分组，互斥选择主题，点击后触发通知；
- 自定义图标 + 红色的 toggle 初始为激活状态，标签在左侧；
- 最后两个分别为紧凑模式和禁用状态，样式和交互各有区分。

2. 绑定数据对象（双向同步）

通过 `bind_value` 实现 toggle 与数据对象的双向绑定，数据变化时组件自动更新，反之亦然：

```
from nicegui import ui

class AppState:
    def __init__(self):
```

```

self.feature_enabled = False
self.selected_language = 'python'

state = AppState()

with ui.column().classes('gap-4'):
    # 绑定布尔值（功能启用状态）
    ui.toggle('Enable Feature').bind_value(state, 'feature_enabled')

    # 绑定多选项（语言选择，分组模式）
    ui.label('Select Language:')
    with ui.row().classes('gap-4'):
        ui.toggle('Python', value='python', group='lang').bind_value(state,
'selected_language')
        ui.toggle('JavaScript', value='js', group='lang').bind_value(state,
'selected_language')
        ui.toggle('Java', value='java', group='lang').bind_value(state, 'selected_language')

    # 显示当前绑定数据（实时同步）
    ui.label().bind_text_from(state, 'feature_enabled', lambda x: f'Feature: {"Enabled" if x
else "Disabled"}')
    ui.label().bind_text_from(state, 'selected_language', lambda x: f'Selected Language:
{x} ')

ui.run()

```

效果：

- 切换第一个 toggle 时，下方标签实时显示功能启用状态；
- 选择不同语言 toggle 时，下方标签实时同步选中的语言；
- 直接修改 `state` 对象的属性（如 `state.feature_enabled = True`），toggle 组件也会自动更新状态。

三、核心功能与进阶用法

1. 多选项分组（radio 模式）进阶

实现带默认值、状态监听的多选项分组，适用于分类选择、模式切换等场景：

```

from nicegui import ui

# 初始选中值
selected_view = 'list'

# 分组 toggle，互斥选择视图模式
with ui.row().classes('gap-3'):
    ui.toggle('List', icon='view_list', value='list', group='view', on_change=lambda e:
update_view(e.value))
    ui.toggle('Grid', icon='view_grid', value='grid', group='view', on_change=lambda e:
update_view(e.value))
    ui.toggle('Cards', icon='view_carousel', value='cards', group='view', on_change=lambda
e: update_view(e.value))

```

```
# 视图切换回调函数
def update_view(view_mode):
    nonlocal selected_view
    selected_view = view_mode
    ui.notify(f'view mode changed to: {view_mode}')

# 初始化默认选中（通过 set_value 手动设置）
ui.toggle(group='view').set_value(selected_view)

ui.run()
```

效果：三个 toggle 组成视图模式选择组，点击切换互斥选项，触发视图更新通知，初始默认选中「List」模式。

2. 自定义样式（颜色、图标、大小）

通过 `color`、`icon`、`classes`、`props` 实现深度样式定制，适配不同界面设计需求：

```
from nicegui import ui

with ui.column().classes('gap-4'):
    # 自定义激活颜色（Tailwind 颜色）
    ui.toggle('Custom color', color='teal-500', value=True)

    # 图标+紧凑模式+圆角样式
    ui.toggle(
        'Icon + Dense + Rounded',
        icon='star',
        dense=True,
        value=True
    ).props('rounded')

    # 禁用状态+自定义文本颜色（通过 classes）
    ui.toggle(
        'Disabled with custom text',
        disabled=True,
        classes='text-gray-500'
    )

    # 标签在上方+大尺寸（通过 props 和 classes）
    ui.toggle(
        'Label on top',
        label_position='top',
        icon='info'
    ).props('size=lg').classes('mt-2')

ui.run()
```

效果：四个 toggle 分别展示自定义颜色、图标 + 紧凑 + 圆角、禁用 + 文本颜色、标签在上 + 大尺寸等样式，覆盖常见定制场景。

3. 数据绑定与状态同步（双向绑定）

结合 `bind_value` 实现组件与数据对象的双向同步，支持动态修改数据或组件状态：

```
from nicegui import ui

class Settings:
    def __init__(self):
        self.notifications = True
        self.dark_mode = False

settings = Settings()

with ui.column().classes('gap-4'):
    # 双向绑定通知开关
    notify_toggle = ui.toggle('Enable notifications').bind_value(settings, 'notifications')

    # 双向绑定暗黑模式
    dark_toggle = ui.toggle('Dark mode', color='black').bind_value(settings, 'dark_mode')

    # 按钮：手动修改数据对象，组件自动同步
    ui.button('Toggle Notifications', on_click=lambda: setattr(settings, 'notifications',
not settings.notifications))

    # 按钮：手动修改组件状态，数据对象自动同步
    ui.button('Toggle Dark Mode', on_click=lambda: dark_toggle.set_value(not
dark_toggle.value))

    # 实时显示数据对象状态
    ui.label().bind_text_from(settings, 'notifications', lambda x: f'Notifications: {x}')
    ui.label().bind_text_from(settings, 'dark_mode', lambda x: f'Dark Mode: {x}')

ui.run()
```

效果：点击组件或按钮修改状态，数据对象与组件自动双向同步，下方标签实时显示当前状态。

4. 事件监听与复杂逻辑处理

通过 `on_change` 监听状态变化，实现复杂业务逻辑（如权限控制、功能联动）：

```
from nicegui import ui

# 功能权限开关
admin_toggle = ui.toggle('Admin Mode', color='red', value=False)

# 依赖 admin 权限的功能开关（初始禁用）
advanced_toggle = ui.toggle('Advanced Features', disabled=True)

# 权限状态变化时联动控制高级功能开关
def on_admin_toggle(e):
    advanced_toggle.set_enabled(e.value) # 启用/禁用高级功能
    if not e.value:
```

```
        advanced_toggle.set_value(False) # 取消 admin 时关闭高级功能
        ui.notify(f'Admin Mode: {"Enabled" if e.value else "Disabled"}')

admin_toggle.on_change(on_admin_toggle)

# 高级功能开关变化监听
advanced_toggle.on_change(lambda e: ui.notify(f'Advanced Features: {"Enabled" if e.value
else "Disabled"}'))

ui.run()
```

效果：未启用 Admin Mode 时，Advanced Features 开关禁用；启用 Admin Mode 后，Advanced Features 开关激活，点击可切换状态；取消 Admin Mode 时，自动关闭 Advanced Features 并禁用。

四、核心属性与方法

1. 常用属性

属性名	类型	说明
classes	Classes[Self]	CSS 类（支持 Tailwind/Quasar 样式）
disabled	BindableProperty	是否禁用（可绑定数据动态控制）
html_id	str	HTML 元素 ID（2.16.0+ 版本支持）
icon	BindableProperty	图标名称（可绑定数据动态修改）
text	BindableProperty	文本标签（可绑定数据动态修改）
value	BindableProperty	当前值（布尔值 / 字符串 / 整数，可绑定数据）
visible	BindableProperty	是否可见（可绑定数据）
color	str	激活状态颜色（支持 Quasar/Tailwind/CSS 颜色）
label_position	str	标签位置（left/right/top/bottom）
dense	bool	是否紧凑模式

2. 关键方法

(1) 状态控制

- `disable()`：禁用组件（不可交互，视觉灰度）
- `enable()`：启用组件
- `set_disabled(value: bool)`：设置禁用状态（True/False）
- `set_visibility(visible: bool)`：设置组件可见性

(2) 属性修改

- `set_value(value: bool | str | int)`: 动态修改组件值 (触发 `on_change` 事件)
- `set_text(text: str)`: 动态修改文本标签
- `set_icon(icon: str | None)`: 动态修改图标
- `set_color(color: str)`: 动态修改激活状态颜色
- `update()`: 触发客户端界面更新 (修改属性后需手动调用, 绑定数据时无需)

(3) 事件与绑定

- `on_change(callback)`: 绑定值变化事件 (`e.value` 为当前值)
- `on(type: str, handler)`: 订阅任意 DOM 事件 (如 `click`、`mousedown`)
- `bind_value(target_object, target_name)`: 双向绑定值到目标对象属性
- `bind_text_from(target_object, target_name)`: 单向绑定文本从目标对象
- `bind_disabled(target_object, target_name)`: 双向绑定禁用状态到目标对象属性

(4) 其他实用方法

- `tooltip(text: str)`: 为组件添加悬停提示
- `delete()`: 彻底删除组件
- `mark(*markers)`: 添加标记 (用于测试或元素查询)
- `clear()`: 移除组件内的所有子元素 (如嵌套的图标、文本)

五、使用场景与注意事项

1. 适用场景

- 布尔值开关: 功能启用 / 禁用 (如通知开关、暗黑模式开关)。
- 多选项分组: 互斥选项选择 (如视图模式、主题、语言选择)。
- 功能联动: 依赖权限或其他状态的功能开关 (如管理员模式下的高级功能)。
- 表单配置: 表单中的二选一或多选项配置项 (如是否接收邮件、数据显示方式)。

2. 关键注意事项

- 分组模式逻辑: 相同 `group` 名称的 `ui.toggle` 自动形成互斥选择 (radio 模式), 此时 `value` 需为字符串 / 整数等非布尔值, 避免冲突。
- 禁用状态行为: 禁用后组件不可点击, 值不会变化, `on_change` 事件不会触发。
- 颜色优先级: `color` 参数仅控制激活状态的颜色 (如开关开启、选项选中), 未激活状态颜色由主题默认样式控制, 可通过 `classes` 自定义。
- 版本兼容性: `html_id` 属性仅在 2.16.0+ 版本支持, `bind_disabled` 的 `strict` 参数在 3.0.0+ 版本支持, 使用时需确认版本。
- 与其他组件的选择:
 - 需单纯布尔开关且样式简洁 → 优先使用 `ui.switch`;

- 需多选项且样式为传统单选框 → 优先使用 `ui.radio`;
- 需多选项分组且样式紧凑、支持图标 → 优先使用 `ui.toggle`。

六、进阶示例：带权限控制的多组 toggle 联动

实现基于用户角色的 toggle 权限控制，不同角色可见 / 可用的 toggle 不同，适用于复杂配置界面：

```
from nicegui import ui

class User:
    def __init__(self, role: str = 'user'):
        self.role = role # 角色: user/admin/super_admin
        self.notifications = True
        self.dark_mode = False
        self.advanced_features = False
        self.system_config = False

# 模拟当前用户（超级管理员）
user = User(role='super_admin')

with ui.column().classes('gap-4'):
    # 基础功能（所有角色可见）
    ui.label('基础功能').classes('font-bold')
    ui.toggle('通知开关').bind_value(user, 'notifications')
    ui.toggle('暗黑模式', color='black').bind_value(user, 'dark_mode')

    # 高级功能（仅 admin 和 super_admin 可见）
    advanced_column = ui.column().classes('gap-2 mt-2')
    with advanced_column:
        ui.label('高级功能').classes('font-bold')
        ui.toggle('高级特性', color='red').bind_value(user, 'advanced_features')
    # 绑定高级功能列的可见性（角色为 admin/super_admin 时显示）
    advanced_column.bind_visibility(
        user, 'role',
        converter=lambda role: role in ['admin', 'super_admin']
    )

    # 系统配置（仅 super_admin 可见且可用）
    system_toggle = ui.toggle('系统配置', color='purple')
    system_toggle.bind_value(user, 'system_config')
    # 绑定系统配置的可见性和禁用状态
    system_toggle.bind_visibility(user, 'role', converter=lambda role: role ==
'super_admin')
    system_toggle.bind_disabled(user, 'role', converter=lambda role: role != 'super_admin')

# 角色切换下拉框（用于测试不同角色权限）
ui.select(
    ['user', 'admin', 'super_admin'],
    value=user.role,
    on_change=lambda e: setattr(user, 'role', e.value)
).classes('mt-4')
```



```
ui.run()
```

效果:

- 普通用户 (user) 仅可见基础功能 toggle;
- 管理员 (admin) 可见基础功能和高级功能 toggle;
- 超级管理员 (super_admin) 可见所有 toggle, 且系统配置 toggle 可交互;
- 切换角色时, 对应 toggle 的可见性和禁用状态自动同步更新。